

C03 -AT3 -MOCK INTERVIEW

Q1 Find

10 marks

What happens if A and B own each other? struct B; struct A { shared_ptr b; }; struct B { shared_ptr<A> a; };

Your Answer

It creates a circular reference, so the reference count never reaches zero and the objects are not destroyed, causing a memory leak.

Q2 Code

10 marks

What happens when this code executes? class Employee { int* salary; public: Employee(int s) { salary = new int(s); } ~Employee() { delete salary; } }; int main() { Employee e1(50000); Employee e2 = e1; }

Your Answer

It causes a double deletion because the default copy makes both objects point to the same salary; when e1 and e2 are destroyed, delete is called twice on the same memory.

Q3 Ambiguity

10 marks

Find the ambiguity. class A { public: void show() { cout << "A"; } }; class B { public: void show() { cout << "B"; } }; class C : public A, public B {}; int main() { C obj; obj.show(); }

Your Answer

The ambiguity is that C inherits two show() functions, one from A and one from B. So obj.show() causes a compile-time ambiguity error.

Fix: obj.A::show(); or obj.B::show();

Q4 Output

10 marks

What is the output? `class Test { public: Test() { cout << "C "; } Test(const Test&) { cout << "Copy "; } ~Test() { cout << "D "; } Test operator+(const Test&) { return Test(); } }; int main() { Test a, b; Test c = a + b; }`

Your Answer

Test a, b; → constructors run: CC
a + b → operator+() creates a temporary Test → C
return Test(); creates the returned object → depending on compiler, copy may be optimized.
At the end, c, b, and a are destroyed → D D D

So the common output is: CCC D D D

Q5 Output

10 marks

What is the output? `class A { public: virtual ~A() { cout << "A "; } }; class B : public A { public: ~B() { cout << "B "; } }; int main() { A* obj = new B(); delete obj; }`

Your Answer

Output:

BA

Explanation: Since the destructor of A is virtual, deleting through the A* pointer first calls B's destructor (B), then A's destructor (A).

Q6 Predict

10 marks

Predict the problem. `struct B; struct A { shared_ptr<B> b; }; struct B { shared_ptr<A> a; };`

Your Answer

It creates a circular reference, causing a memory leak because the shared_ptr objects keep each other alive and their reference counts never become zero.

Q7 Constructor

10 marks

Which constructor is called? `class Test { public: Test() { cout << "Default"; } Test(int x) { cout << "Parameterized"; } Test(const Test&) { cout << "Copy"; } }; int main() { Test a; Test b(10); Test c = b; }`

Your Answer

The constructors called are:

Test a; → Default

Test b(10); → Parameterized

Test c = b; → Copy

Output:

Q10 Operator

10 marks

Which increment operation does this represent, and what would you change for postfix `counter++`?

Your Answer

It represents the prefix increment (`++counter`). For postfix `counter++`, change the operator to `operator++(int)` and return the old value before incrementing.